

**ECE 759 - High Performance Computing
Project Proposal**

**Abhinarayani Velu Radhakrishnan
Sreeraj Kannakarankodi**

Characterizing Software-Directed Prefetching for Irregular SpMV Workloads

1. Motivation

The fundamental motivation for this project is the growing "Memory Wall"—the disparity between processor speed and memory latency. In modern architectures, fetching data from DRAM can take hundreds of cycles, during which the processor sits idle. While hardware prefetchers excel at predicting linear patterns, they frequently fail with irregular workloads like Sparse Matrix-Vector Multiplication (SpMV) because the memory access is data-dependent ($x[\text{col_idx}[j]]$).

As a computer architect, my goal is to explore the Hardware-Software Contract. By using software "intelligence" to issue prefetch hints, we can potentially hide memory latency that hardware logic alone cannot detect. This is critical for modern applications in graph analytics, social network modeling, and AI recommendation systems, where irregular data structures are the primary bottleneck to efficiency and throughput. We will be targeting the use case of Graph Neural Networks (GNNs), specifically for Fraud Detection or Recommendation Engines. In a GNN, each "node" (user) gathers information from its "neighbors" to decide if it's a fraudster. This "gathering" is mathematically an SpMV operation.

2. Objective

The goal of this project is to quantify the effectiveness of Software-Directed Prefetching in mitigating memory-bound stalls. We will implement and profile SpMV using the Compressed Sparse Row (CSR) format to determine how manual prefetching hints (via OpenMP and CUDA) improve cache hit rates and overall Instructions Per Cycle (IPC) on the Euler HPC cluster.

3. Proposed Methodology

Phase A: CPU Optimization (OpenMP)

- Baseline: Implement a standard parallel SpMV using `omp parallel for` work-sharing.
- Implementation: Integrate the `__builtin_prefetch` compiler intrinsic to pre-stage data into the L1/L2 cache hierarchy.
- Architectural Analysis: Conduct a parametric study on Prefetch Distance (k) to find the optimal balance between latency hiding and "Cache Pollution" (prematurely evicting useful data).

Phase B: GPU Optimization (CUDA)

- Baseline: Develop a CUDA kernel where each thread processes one row of the matrix using standard global memory access.

- Implementation: Utilize Warp-Level Primitives (e.g., `__shfl_sync`) and Shared Memory to coordinate collective data fetching within a warp.
- Architectural Analysis: Analyze how Global Memory Access Patterns and Warp-Level Divergence affect the efficiency of manual prefetching into the GPU's shared memory.

4. Profiling and Performance Metrics

To validate the architectural impact, we will use these tools to gather hardware-level data:

- Linux `perf`: To measure CPU L1-dcache-load-misses and LLC (Last Level Cache) misses on the Euler cluster.
- NVIDIA Nsight Compute (`ncu`): To analyze GPU memory workload, specifically the Cache Hit Rate and Memory Throughput.
- Scaling Analysis: Generate plots comparing baseline performance vs. prefetched performance across various matrix densities and sizes.
- **Execution Time vs. Sparsity Pattern:** Use matrices from the [SuiteSparse Matrix Collection](#). Compare a "regular" matrix (like a 2D grid) where the prefetcher does nothing, to a "Power Law" matrix (like a Twitter graph) where the prefetcher should give a **15–30% speedup**.
- **The Lookahead Sweep:** Create a graph of **Lookahead Distance (1 to 64)** vs. **Cycles per Non-Zero (CPNNZ)**. This will visually identify the Sweet Spot mentioned in the proposal.